

ATIVIDADE PRÁTICA — MVC, JDBC e BANCO DE DADOS

Prof. Guilherme Vicentini

ABORDAGEM

Ao longo das últimas aulas, construímos juntos um projeto Java completo: partimos das dores de um cliente, levantamos os requisitos, organizamos o código no padrão MVC, criamos a camada de repositório com SQL puro via JDBC e conectamos tudo a um banco de dados PostgreSQL real.

Agora é a sua vez de percorrer esse mesmo caminho de forma autônoma. Você receberá três cenários diferentes a seguir, cada um com um cliente relatando seus problemas, e deverá ir do levantamento das entidades e regras até um sistema funcional, com banco de dados, CRUD completo e a simulação de um movimento pela classe Main.

Para cada cenário, siga as mesmas instruções abaixo de como deverá ficar a estrutura de cada projeto de vocês e a separação em branches no Github. No final, postem o link do trabalho.

COMO ORGANIZAR O PROJETO

Para todo o trabalho, crie apenas um único repositório público no GitHub e para cada cenário/exercício, crie uma Branch dedicada para ele:

Branch “cenário1” → versão final e completa com MVC, JDBC e PostgreSQL conectado **para o exercício 1**.

Branch “cenário2” → versão final e completa com MVC, JDBC e PostgreSQL conectado **para o exercício 2**.

Branch “cenário3” → versão final e completa com MVC, JDBC e PostgreSQL conectado **para o exercício 3**.

Para o seu projeto Java, crie um novo projeto no **IntelliJ**, selecione a opção **Maven** em *Build system* e em *Advanced Settings* e crie um **GroupId** e **ArtifactId** conforme fizemos em aula. Na sua pasta principal do novo projeto criado, a estrutura de pacotes esperada para cada cenário é o padrão **MVC** que viemos trabalhando em sala de aula:

model → classes que representam as entidades;
repository → uma classe por entidade com os métodos CRUD usando SQL via JDBC;
service → regras de negócio, validações e orquestração entre repositories;
controller → recebe a ação, chama o service e exibe ou retorna o resultado;
util → classe Conexao.java com a configuração da conexão ao banco;
Main.java → simula o fluxo completo: cadastros simples + movimento;

Na raiz do seu projeto, crie um arquivo README.md para cada cenário que deva conter obrigatoriamente dentro dele:

- Tabelas identificadas com os campos mínimos de cada uma
- Regras de negócio levantadas a partir do texto do cliente

O QUE ENTREGAR

1. Um único repositório público no GitHub que representa o trabalho completo;
2. Uma Branch para cada cenário/exercício, com: MVC + JDBC + PostgreSQL + simulação do movimento na Main
3. Arquivo README.md com tabelas identificadas para cada cenário e suas regras de negócio

No envio da atividade, coloque o Nome e RA da equipe e cole o link do repositório!

CENÁRIO 1 – Sistema de Clínica Veterinária

A dor do cliente:

Eu tenho uma clínica veterinária pequena e ainda controlo tudo em fichas de papel. O problema é que quando um tutor liga para saber o histórico do pet dele, eu preciso procurar manualmente entre várias fichas, que às vezes nem encontro. Já aconteceu de eu registrar uma consulta no nome errado do animal porque tem dois pets com o mesmo nome, mas com donos diferentes.

Eu preciso de um sistema onde eu possa cadastrar os tutores (donos dos animais) informando pelo menos nome, endereço e telefone. Também preciso cadastrar os animais de cada tutor, com pelo menos nome, espécie e raça. Um tutor pode ter mais de um animal cadastrado.

O sistema precisa registrar as consultas realizadas. Cada consulta deve informar qual animal foi atendido, a data do atendimento, o motivo do atendimento e o valor cobrado. Não quero permitir registrar uma consulta para um animal que não esteja cadastrado, e o valor da consulta não pode ser negativo.

Também preciso conseguir consultar todas as consultas de um animal específico, para ter o histórico de atendimentos. E se um tutor me ligar novamente, eu quero ter a possibilidade de ver todos os animais que ele tem no sistema.

O que se pede:

1. Leia o texto do cliente, identifique as entidades necessárias e os campos mínimos de cada uma. Registre os comandos **CREATE TABLE** necessários, registrando o comando SQL no **README.md**;
2. Identifique todas as regras de negócio descritas pelo cliente e registre também no **README.md**;
3. Na branch cenário1: implemente **MVC + JDBC + PostgreSQL** com as tabelas e relacionamentos corretos. Implemente o CRUD de cada entidade nos seus respectivos repositories. Simule na Main o fluxo: **tutor** → **animal** → **consulta**.

Dica sobre o movimento:

A consulta é o movimento deste exercício. Ela não existe por si só, pois depende de um animal cadastrado, que por sua vez depende de um tutor cadastrado. Na simulação da Main, demonstre esse encadeamento completo: primeiro crie o tutor, depois o animal vinculado a ele, depois registre a consulta.

Estrutura mínima de tabelas sugerida:

Entidade	Campos mínimos sugeridos	Observação
Tutor	id, nome, endereço, telefone	Cadastro simples — existe independente
Animal	id, nome, especie, raca, id_tutor	Cadastro simples — vinculado ao tutor
Consulta	id, id_animal, data, motivo, valor	Movimento — depende do animal (e indiretamente do tutor)

CENÁRIO 2 – Sistema de Oficina Mecânica

A dor do cliente:

Eu tenho uma oficina mecânica e hoje anoto tudo à mão em um caderno. Quando um cliente volta pela segunda vez, eu preciso lembrar ou perguntar de novo qual é o carro dele, o que foi feito antes, quanto ele deve. É uma bagunça!!

Eu preciso de um sistema onde eu cadastre os clientes informando pelo menos nome e telefone. Cada cliente pode ter mais de um veículo, então também preciso registrar os veículos com pelo menos placa, modelo e ano. O que eu mais preciso é conseguir abrir uma ordem de serviço. Quando um carro entra na oficina, quero registrar qual veículo é, descrever o problema relatado pelo cliente, informar o valor do serviço e definir se a ordem está aberta ou já foi concluída.

Não quero permitir abrir uma ordem de serviço para um veículo que não esteja cadastrado. O valor do serviço não pode ser negativo. E preciso conseguir consultar todo o histórico de manutenções de um determinado veículo.

O que se pede:

1. Leia o texto do cliente, identifique as entidades necessárias e os campos mínimos de cada uma. Registre os comandos **CREATE TABLE** necessários, registrando o comando SQL no **README.md**;
2. Identifique todas as regras de negócio descritas pelo cliente e registre também no **README.md**;
3. Na branch cenário1: implemente **MVC + JDBC + PostgreSQL** com as tabelas e relacionamentos corretos. Implemente o CRUD de cada entidade nos seus respectivos repositories. Simule na Main o fluxo: **cliente** → **veículo** → **ordem de serviço**.

Dica sobre o movimento:

A ordem de serviço é o movimento deste exercício. Ela não existe sem um veículo cadastrado, e o veículo não existe sem um cliente. Na simulação da Main, demonstre esse encadeamento: primeiro cadastre o cliente, depois o veículo, depois abra a ordem de serviço para esse veículo.

Estrutura mínima de tabelas sugerida:

Entidade	Campos mínimos sugeridos	Observação
cliente	id, nome, telefone	Cadastro simples — existe independente
veiculo	id, placa, modelo, ano, id_cliente	Cadastro simples — vinculado ao cliente
ordem_servico	id, id_veiculo, descricao, valor, status	Movimento — depende do veículo (e indiretamente do cliente)

CENÁRIO 3 – Sistema de Escola de Cursos Livres

A dor do cliente:

Eu tenho uma escola de cursos livres e controlo as matrículas em planilhas. O problema é que às vezes a mesma pessoa acaba matriculada duas vezes no mesmo curso porque alguém esqueceu de verificar antes, ou eu acabo abrindo matrícula em um curso que já não tem mais vagas disponíveis.

Eu preciso de um sistema onde eu cadastre os alunos com pelo menos nome, e-mail e telefone. Também preciso cadastrar os cursos com pelo menos nome, descrição, carga horária e número máximo de vagas. O sistema precisa registrar a matrícula de um aluno em um curso, informando a data e o valor pago. Não quero que seja possível matricular um aluno em um curso que já atingiu o limite de vagas. Também não quero que o mesmo aluno seja matriculado duas vezes no mesmo curso.

Preciso conseguir consultar todos os alunos matriculados em um curso e também todos os cursos em que um aluno está matriculado. O valor pago na matrícula não pode ser negativo, e não posso matricular um aluno que não esteja cadastrado nem em um curso que não esteja cadastrado.

O que se pede:

1. Leia o texto do cliente, identifique as entidades necessárias e os campos mínimos de cada uma. Registre os comandos **CREATE TABLE** necessários, registrando o comando SQL no **README.md**;
2. Identifique todas as regras de negócio descritas pelo cliente e registre também no **README.md**;
3. Na branch cenário1: implemente **MVC + JDBC + PostgreSQL** com as tabelas e relacionamentos corretos. Implemente o CRUD de cada entidade nos seus respectivos repositories. Simule na Main o fluxo: **aluno** → **curso** → **matrícula**. Tente também simular uma matrícula inválida (sem vaga ou duplicada) para demonstrar que as regras estão funcionando.

Dica sobre o movimento:

A matrícula é o movimento deste exercício. Ela conecta um aluno a um curso e só pode existir se ambos já estiverem cadastrados. Além disso, ela tem regras próprias: não pode exceder o número de vagas do curso, nem matricular o mesmo aluno duas vezes. Essas validações devem ficar no service.

Estrutura mínima de tabelas sugerida:

Entidade	Campos mínimos sugeridos	Observação
aluno	id, nome, email, telefone	Cadastro simples — existe independente
curso	id, nome, descricao, carga_horaria, vagas_totais, vagas_disponiveis	Cadastro simples — existe independente
matricula	id, id_aluno, id_curso, data_matricula, valor	Movimento — depende do aluno E do curso simultaneamente